

GST Purchase & Sales Billing Software — Technical Specification

Prepared for: Development handoff to Claude Code **Stack:** MERN (MongoDB, Express.js, React, Node.js) — two independent repos (`/backend` , `/frontend`) **Deployment target:** Backend → Render, Frontend → Vercel, DB → MongoDB Atlas **Reference document:** Sample GST Tax Invoice (Patel Flexible → K.K. Enterprise)

1. Project Objective

Build a purchase & sales billing web application that lets the client:

- Create GST-compliant tax invoices for **sales** (matching the format of the reference sample) and record GST-compliant **purchase bills** with the same depth — own numbering series, own PDF, own dynamic fields
 - Manage buyers/sellers (parties), products, and recurring invoice/purchase numbering
 - Auto-calculate CGST/SGST/IGST based on buyer/supplier state vs. seller state, for both sales and purchase documents
 - Auto-trigger e-way bill generation workflow when a sales invoice or purchase bill crosses the statutory threshold (₹50,000 taxable value, configurable)
 - Generate printable/downloadable PDFs for both sales invoices and purchase bills
 - View purchase and sales registers plus basic GST reports across both
-

2. Reference Invoice — Field Breakdown

This is the ground truth for the Invoice schema and PDF layout. Every field below must be represented.

Header

- Document title: "Tax Invoice" + copy type (Original/Duplicate for Transporter/Triplicate for Supplier)

- Invoice No. (format: GST-<sequence> , auto-incrementing per financial year)
- Invoice Date
- Delivery Note, Mode/Terms of Payment
- Supplier's Ref, Other Reference(s)
- Buyer's Order No. + Date
- Despatch Document No. + Delivery Note Date
- Despatched Through, Destination
- Terms of Delivery

Seller block

- Company name, address, GSTIN/UIN, State Name + State Code

Buyer block

- Buyer name, address, GSTIN/UIN, State Name + State Code

Line items table

- Sl No, Description of Goods, HSN/SAC, Quantity + unit, Rate, per-unit, Amount

Tax block

- CGST amount, SGST amount (or IGST if inter-state), Round Off
- Total quantity, Total amount

Summary

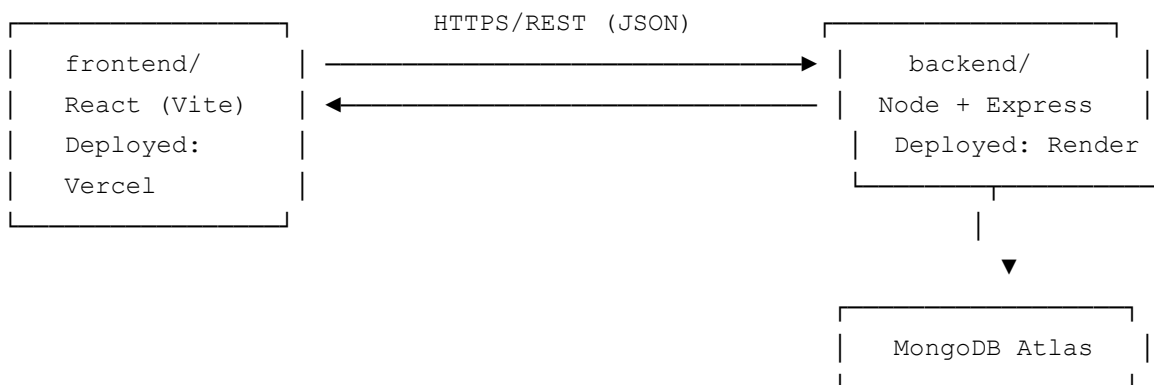
- Amount Chargeable (in words)
- HSN/SAC-wise tax breakup table: Taxable Value, CGST Rate/Amount, SGST Rate/Amount, Total Tax Amount
- Tax Amount (in words)
- Company's PAN
- Company's Bank Details: Bank Name, A/c No, Branch & IFSC Code
- Declaration text
- Authorised Signatory (signature image upload)
- "This is a Computer Generated Invoice" footer

GST logic to encode:

- If Buyer State Code == Seller State Code → apply CGST + SGST (split tax rate, e.g., 9% + 9%)
- If different → apply IGST (full rate, e.g., 18%)
- Round Off = $\text{round}(\text{Total}) - \text{Total}$, shown as a signed adjustment line

Purchase bills mirror this exact structure, with roles reversed: the client's own company becomes the "Buyer" block and the supplier becomes the "Seller" block. Same line-items table, same CGST/SGST/IGST logic, same round-off — recorded as a `PurchaseBill` document (Section 6) with its own numbering series and PDF layout, since it represents a bill the client *received*, not one they issued.

3. High-Level Architecture



- Stateless REST API, JWT-based auth (single business owner + optional staff roles: Admin, Accountant)
 - PDF generation happens server-side (Puppeteer or pdf-lib) so the invoice always renders identically regardless of client
 - File storage (signature image, company logo) via Cloudinary or local `/uploads` initially — Cloudinary preferred for Render's ephemeral filesystem
-

4. Backend — Folder Structure (`/backend`)

Per your direction: `index.js` is the single entry point, `router.js` is the single file holding every route (mapped straight to controllers), and `methods.js` is a centralized module that

every controller calls into for DB operations — controllers never touch Mongoose models directly.

```
backend/
├─ config/
│   ├── db.js                # Mongoose connection
│   ├── env.js               # centralized env var loader/validator
│   └─ cloundinary.js
├─ models/
│   ├── Company.model.js     # seller's own company profile (multi-company
│   ├── Party.model.js       # buyers/suppliers
│   ├── Product.model.js     # HSN/SAC, default rate, tax rate, unit
│   ├── Invoice.model.js      # sales invoice
│   ├── PurchaseBill.model.js # purchase entries
│   ├── Counter.model.js     # atomic invoice-number sequence per FY
│   ├── EwayBill.model.js
│   ├── FieldConfig.model.js # NEW - dynamic/custom field definitions (see
│   ├── Master.model.js      # NEW - generic Tally-style masters (units, t
│   └─ User.model.js
├─ methods.js                # CENTRALIZED DB OPERATIONS - every generic M
│                             # (create, findAll, findById, updateById, de
│                             # aggregate helpers) lives here, parameteri
│                             # Controllers only ever call methods.js - ne
├─ controllers/
│   ├── auth.controller.js
│   ├── company.controller.js
│   ├── party.controller.js
│   ├── product.controller.js
│   ├── invoice.controller.js
│   ├── purchase.controller.js
│   ├── ewaybill.controller.js
│   ├── report.controller.js
│   ├── fieldConfig.controller.js # NEW - CRUD for dynamic field/master definit
│   └─ template.controller.js    # NEW - list templates, set company template
├─ router.js                 # SINGLE ROUTES FILE - every route declared h
│                             # PROJECT CONVENTION: every route is POST -
│                             # anywhere in the API. Filters, IDs, and pag
│                             # normally be query params or path params ar
│                             # request body instead. Each route's handler
│                             # a controller function, mounted under /api/
├─ services/
│   ├── gst.service.js        # CGST/SGST/IGST + round-off calculation
│   ├── invoiceNumber.service.js
│   ├── pdf.service.js        # Puppeteer/pdf-lib invoice rendering
│   ├── numberToWords.service.js
│   └─ ewaybillThreshold.service.js
```

```

├── dynamicField.service.js      # NEW - validates/merges customFields against
├── middlewares/
│   ├── auth.middleware.js      # JWT verify
│   ├── role.middleware.js
│   ├── error.middleware.js     # centralized error handler
│   ├── validate.middleware.js  # zod/joi schema validation
│   └── rateLimiter.middleware.js
├── validators/
│   ├── invoice.validator.js
│   ├── party.validator.js
│   └── product.validator.js
├── utils/
│   ├── ApiError.js
│   ├── ApiResponse.js
│   ├── asyncHandler.js
│   └── logger.js               # winston/pino
├── templates/
│   ├── invoice/                # NEW - three selectable layouts, see Section
│   │   ├── classic.template.html # matches the reference sample exactly (default)
│   │   ├── modern.template.html
│   │   └── detailed.template.html
│   └── purchase/               # NEW - same three layouts, roles swapped
│       ├── classic.template.html
│       ├── modern.template.html
│       └── detailed.template.html
├── app.js                      # express app, middleware wiring, mounts routes
├── index.js                    # ENTRY POINT - connects DB, starts HTTP server
├── .env.example
├── package.json
└── render.yaml (optional)

```

methods.js — centralized DB layer (why this matters): instead of every controller calling `Model.find()`, `Model.findByIdAndUpdate()`, etc. directly, `methods.js` exposes generic, reusable functions that take the `Model` as a parameter:

```

// methods.js (shape, not final code)
export const create      = (Model, data) => Model.create(data);
export const findAll     = (Model, filter = {}, options = {}) => Model.find(filter, options);
export const findById    = (Model, id) => Model.findById(id).lean();
export const findOne     = (Model, filter) => Model.findOne(filter).lean();
export const updateById  = (Model, id, data) => Model.findByIdAndUpdate(id, data, { overwrite: true });
export const deleteById  = (Model, id) => Model.findByIdAndDelete(id);
export const paginate    = (Model, filter, page = 1, limit = 20) => { /* skip/limit logic */ };
export const aggregate   = (Model, pipeline) => Model.aggregate(pipeline);

```

Every controller becomes: `import * as methods from '../methods.js'; const doc = await methods.create(Invoice, req.body);` — one place to add logging, caching, soft-delete filters, or audit trails later without touching every controller.

router.js — single routes file, POST-only convention (shape, not final code):

```
// router.js
// PROJECT CONVENTION: every route is POST. No GET/PUT/PATCH/DELETE anywhere.
// IDs, filters, and pagination all travel in req.body instead of the URL.
import express from 'express';
import authMiddleware from './middlewares/auth.middleware.js';
import * as authController from './controllers/auth.controller.js';
import * as partyController from './controllers/party.controller.js';
import * as invoiceController from './controllers/invoice.controller.js';
import * as fieldConfigController from './controllers/fieldConfig.controller.js';
// ...import remaining controllers

const router = express.Router();

router.post('/auth/login', authController.login);
router.post('/auth/register', authController.register);

router.post('/parties/list', authMiddleware, partyController.list);      // filt
router.post('/parties/create', authMiddleware, partyController.create);
router.post('/parties/update', authMiddleware, partyController.update);  // { id
router.post('/parties/delete', authMiddleware, partyController.remove);  // { id

router.post('/invoices/create', authMiddleware, invoiceController.create);
router.post('/invoices/list', authMiddleware, invoiceController.list);
router.post('/invoices/detail', authMiddleware, invoiceController.detail); // { i
router.post('/invoices/pdf', authMiddleware, invoiceController.downloadPdf); // {
router.post('/invoices/update', authMiddleware, invoiceController.update);
router.post('/invoices/delete', authMiddleware, invoiceController.remove);

// field-config / masters (Tally-style dynamic setup — see Section 6a)
router.post('/field-configs/list', authMiddleware, fieldConfigController.list);
router.post('/field-configs/create', authMiddleware, fieldConfigController.create
router.post('/masters/list', authMiddleware, fieldConfigController.listMasters);
router.post('/masters/create', authMiddleware, fieldConfigController.createMaster

export default router;
```

index.js — entry point (shape, not final code):

```
// index.js
```

```
import app from './app.js';
import connectDB from './config/db.js';
import { validateEnv } from './config/env.js';

validateEnv();
connectDB().then(() => {
  app.listen(process.env.PORT, () => console.log(`Server running on ${process.env
}));
```

Project-wide note: set "type": "module" in backend/package.json so Node treats .js files as ES modules. Every local import needs the explicit .js extension (e.g. ./config/db.js, not ./config/db) — this is required by native ESM in Node and differs from the extension-optional CommonJS style. Third-party packages that only ship CommonJS (occasionally relevant for older Puppeteer plugins) still work fine via import — Node handles the interop automatically.

```
**Why services/ still exists alongside methods.js:** `methods.js` is generic Mong
```

```
---
```

```
## 5. Frontend — Folder Structure (`/frontend`)
```

```
frontend/ ├── src/ | ├── api/ || ├── axiosClient.js # base instance, interceptors, token
refresh || ├── auth.api.js || ├── party.api.js || ├── product.api.js || ├──
invoice.api.js || ├── purchase.api.js || ├── report.api.js | ├── assets/ | ├──
components/ || ├── common/ # Button, Input, Table, Modal, Loader || ├── layout/ #
Sidebar, Navbar, DashboardLayout || ├── invoice/ || ├── InvoiceForm.jsx || ├──
InvoiceLineItems.jsx || ├── InvoicePreview.jsx # mirrors PDF layout on-screen || ├──
TaxSummaryTable.jsx || ├── purchase/ || ├── PurchaseForm.jsx # NEW — mirrors
InvoiceForm, roles reversed (supplier instead of buyer) || ├── PurchaseLineItems.jsx #
NEW — reuses TaxSummaryTable logic || ├── PurchasePreview.jsx # NEW — mirrors on-
screen preview, feeds purchase PDF || ├── PurchaseAttachmentUpload.jsx # NEW —
upload scanned/photographed supplier bill | ├── pages/ || ├── auth/Login.jsx || ├──
dashboard/Dashboard.jsx || ├── parties/PartyList.jsx || ├── parties/PartyForm.jsx ||
| ├── products/ProductList.jsx || ├── products/ProductForm.jsx || ├──
invoices/InvoiceList.jsx || ├── invoices/CreateInvoice.jsx || ├──
invoices/InvoiceDetail.jsx || ├── purchases/PurchaseList.jsx || ├──
purchases/CreatePurchase.jsx # NEW — mirrors CreateInvoice || ├──
purchases/PurchaseDetail.jsx # NEW — mirrors InvoiceDetail, includes attachment viewer ||
| ├── ewaybill/EwayBillTracker.jsx || ├── reports/Reports.jsx || ├── settings/ || ├──
```

ManageFields.jsx # NEW — add/reorder/deactivate custom fields per entity || |——
 ManageMasters.jsx # NEW — Tally-style master lists (units, tax rates, voucher types) ||
 |—— TemplateGallery.jsx # NEW — pick invoice/purchase layout, see Section 16 | |——
 components/settings/ || |—— TemplatePreviewCard.jsx # NEW — thumbnail + live preview
 for one template option | |—— components/dynamic/ || |—— DynamicFieldRenderer.jsx #
 NEW — renders text/number/date/dropdown from FieldConfig || |—— FieldConfigForm.jsx
 # NEW — form to create/edit a FieldConfig entry | |—— hooks/ || |—— useAuth.js ||
 |—— useInvoiceCalculations.js # live CGST/SGST/round-off calc as user types || |——
 useFieldConfig.js # NEW — fetches active FieldConfig list for an entity || |——
 useDebounce.js | |—— context/ || |—— AuthContext.jsx | |—— store/ # Redux Toolkit or
 Zustand slices || |—— authSlice.js || |—— invoiceSlice.js || |—— index.js | |—— utils/ |
 | |—— gstCalculator.js # mirrors backend logic for instant UI feedback || |——
 numberToWords.js || |—— formatters.js # currency, date | |—— routes/ || |——
 AppRoutes.jsx || |—— ProtectedRoute.jsx | |—— App.jsx | |—— main.jsx |——
 .env.example |—— vite.config.js |—— package.json

6. Core Database Schemas (MongoDB / Mongoose)

****Company**** — one document per business using the software (supports future multi
 - name, address, gstin, stateName, stateCode, pan, bankDetails{ bankName, account
 - invoiceTemplate, purchaseTemplate (`classic` | `modern` | `detailed` — see Sect

****Party**** (buyer/supplier)
 - name, address, gstin, stateName, stateCode, type (buyer/supplier/both), contact
 - `customFields: { type: Map, of: Mixed }` — see Section 6a

****Product****
 - name, hsnSac, unit (ref Master:unit), defaultRate, gstRate (ref Master:taxRate)
 - `customFields: { type: Map, of: Mixed }` — see Section 6a

****Counter**** (for atomic, gap-free numbering per financial year — shared by both s
 - key (e.g. `INV-2026-27`, `PUR-2026-27`), seq, prefix (client-editable, Tally-st

****Invoice**** (sales — outgoing)
 - invoiceNo (generated via Counter), invoiceDate, company (ref, = seller), buyer
 - items: [{ product (ref), description, hsnSac, quantity, unit, rate, amount, cus
 - taxableValue, cgstRate, cgstAmount, sgstRate, sgstAmount, igstRate, igstAmount,
 - amountInWords, taxAmountInWords
 - ewayBillRequired (boolean, auto-set by threshold service), ewayBillNo
 - status (draft/finalized/cancelled), createdBy (ref User)
 - `customFields: { type: Map, of: Mixed }` — see Section 6a

- templateOverride (optional, `classic` | `modern` | `detailed` – see Section 16)

****PurchaseBill**** (purchase – incoming, full parity with Invoice)

- purchaseNo (generated via Counter, own `PUR-<FY>` series), purchaseDate
- company (ref, = the client's own company – plays the "Buyer" role here)
- supplier (ref Party, `type: supplier`) – plays the "Seller" role
- supplierInvoiceNo, supplierInvoiceDate – the bill number/date as printed on the
- items: [{ product (ref), description, hsnSac, quantity, unit, rate, amount, cus
- taxableValue, cgstRate, cgstAmount, sgstRate, sgstAmount, igstRate, igstAmount,
- amountInWords, taxAmountInWords
- ewayBillRequired (boolean, auto-set by threshold service), ewayBillNo (supplier
- paymentStatus (unpaid/partial/paid), amountPaid
- status (draft/finalized/cancelled), createdBy (ref User)
- attachmentUrl – scanned copy/photo of the supplier's physical bill (Cloudinary)
- `customFields: { type: Map, of: Mixed }` – see Section 6a
- templateOverride (optional, `classic` | `modern` | `detailed` – see Section 16)

****EwayBill****

- invoice (ref Invoice, nullable) OR purchaseBill (ref PurchaseBill, nullable) –
- ewbNo, generatedDate, validUpto, vehicleNo, transporterName, distance

****User****

- name, email, passwordHash, role (admin/accountant), company (ref)

****FieldConfig**** (NEW – drives dynamic fields, replaces hardcoded form fields)

- entity (`Party` | `Product` | `Invoice` | `PurchaseBill` | `InvoiceItem`)
- fieldKey (e.g. `transportMode`), label (e.g. "Mode of Transport")
- fieldType (`text` | `number` | `date` | `dropdown` | `boolean`)
- options: [String] (for dropdown type)
- isRequired, isSystemField (system fields like `name`/`gstIn` are locked, not de
- displayOrder, isActive
- company (ref – field configs are per-company, so each client can shape their ow

****Master**** (NEW – generic Tally-style master data: replaces hardcoded enums)

- type (`unit` | `taxRate` | `voucherType` | `paymentTerms` | `state`)
- code, label, value (e.g. type=`taxRate`, code=`GST18`, value=`18`)
- company (ref), isActive

Indexes: `Invoice.invoiceNo` (unique per company+FY), `PurchaseBill.purchaseNo` (

6a. Dynamic Fields – Tally Replacement Architecture

Your client is moving off Tally, where every master (Ledger, Stock Item, Unit, Vo

****How it works end-to-end:****

1. **Core/system fields are fixed** (name, GSTIN, invoice number, tax amounts – a
2. **Everything else is a custom field**, stored in a `customFields`` Map on the r
3. When the client goes to **Settings → Manage Fields** (Party / Product / Invoice
 - Add a new field (label, type, required?, dropdown options)
 - Reorder fields (drag and drop, saved as `displayOrder``)
 - Deactivate a field (soft-remove – historical data stays intact, field just s
4. **Frontend** fetches field definitions via `POST /field-configs/list`` (body: ```
5. **Backend** (`dynamicField.service.js``) validates incoming `customFields`` agai
6. **Masters** (units, tax rates, voucher types, payment terms) work the same way
7. **PDF template** reads `FieldConfig.displayOrder` + `label`` at render time so

Practical scope note: full "dynamic schema" (client redefining the invoice's

7. Key API Endpoints (v1)

Project convention: every endpoint is `POST``. No GET/PUT/PATCH/DELETE anywhere

`POST /api/v1/auth/login` `POST /api/v1/auth/register`

`POST /api/v1/parties/list` # body: { page, limit, search, type } `POST /api/v1/parties/create`
`POST /api/v1/parties/update` # body: { id, ...fields } `POST /api/v1/parties/delete` # body: { id }

`POST /api/v1/products/list` `POST /api/v1/products/create` `POST /api/v1/products/update`
`POST /api/v1/products/delete`

`POST /api/v1/invoices/create` # runs `gst.service + invoiceNumber.service` `POST`
`/api/v1/invoices/list` # body: { page, limit, from, to, party, status } `POST /api/v1/invoices/detail`
body: { id } `POST /api/v1/invoices/pdf` # body: { id } — streams generated PDF `POST`
`/api/v1/invoices/update` # body: { id, ...fields } `POST /api/v1/invoices/delete` # body: { id } —
soft delete/cancel only, never hard-delete finalized invoices

`POST /api/v1/purchases/list` # body: { page, limit, from, to, supplier, status } `POST`
`/api/v1/purchases/create` # runs `gst.service + invoiceNumber.service` (PUR- series) `POST`
`/api/v1/purchases/detail` # body: { id } `POST /api/v1/purchases/pdf` # body: { id } — streams
generated purchase bill PDF `POST /api/v1/purchases/update` # body: { id, ...fields } `POST`
`/api/v1/purchases/delete` # body: { id } — soft delete/cancel only, never hard-delete finalized
bills

`POST /api/v1/ewaybills/list` `POST /api/v1/ewaybills/create` # manual entry of e-way bill
number once generated on govt portal

`POST /api/v1/reports/sales-register` # body: { from, to } `POST /api/v1/reports/purchase-`

register # body: { from, to } POST /api/v1/reports/gst-summary # body: { from, to } — combines both sales and purchase for input/output tax

POST /api/v1/field-configs/list # body: { entity } — Party | Product | Invoice | PurchaseBill | InvoiceItem POST /api/v1/field-configs/create # add a custom field POST /api/v1/field-configs/update # body: { id, ...fields } — rename/reorder/toggle active POST /api/v1/field-configs/delete # body: { id } — blocked for isSystemField=true

POST /api/v1/masters/list # body: { type } — unit | taxRate | voucherType | paymentTerms POST /api/v1/masters/create # client adds their own master entry (Tally-style) POST /api/v1/masters/update # body: { id, ...fields } POST /api/v1/masters/delete # body: { id }

POST /api/v1/templates/list # body: { documentType } — invoice | purchase — lists available templates POST /api/v1/companies/set-template # body: { companyId, invoiceTemplate, purchaseTemplate }

Note: Actual government e-Way Bill API / GSTN integration requires GSP (GST Suvid

8. Code Quality & Optimization Guidelines (for Claude Code to follow)

Backend

- Use `asyncHandler` wrapper on every controller — no repeated try/catch blocks
- Centralized error middleware returns consistent `{ success, message, errors }`
- Validate all incoming payloads with Joi/Zod before hitting controllers
- Use `.lean()` on read-only Mongoose queries for performance
- Paginate all list endpoints (`page`, `limit` in the POST request body — see Sec
- Cache PDF templates in memory; don't re-read the HTML template file per request
- Use MongoDB transactions when creating an invoice + incrementing the Counter to
- Environment variables validated at boot (fail fast if `MONGO\_URI`/`JWT\_SECRET`

Frontend

- Code-split routes with `React.lazy` + `Suspense`
- Debounce live tax calculations on line-item input
- Keep `InvoicePreview` as a pure presentational component reused by both the on-
- Central Axios instance with interceptor for auth token + 401 redirect
- Form validation with react-hook-form + zod resolver (avoid re-renders on every
- Memoize computed tax summaries (`useMemo`) in the invoice form

General

- **Module system: ES Modules (`import`/`export`) throughout the backend — no `re
- Consistent ESLint + Prettier config in both repos
- `.env.example` committed in both repos, `.env` gitignored
- Meaningful commit structure if Claude Code is asked to scaffold incrementally:

9. Security Considerations

This handles a client's GSTIN, bank account details, and financial records – trea

Authentication & authorization

- Passwords hashed with bcrypt (cost factor ≥ 10), never stored or logged in plain
- JWT access tokens short-lived (e.g. 1 hour) with a longer-lived refresh token s
- ``role.middleware.js`` enforces Admin vs Accountant permissions at the route leve
- Every data-fetching query scoped by ``company`` (from the authenticated user's to

Input handling

- All request bodies validated with Joi/Zod (``validate.middleware.js``) before rea
- ``dynamicField.service.js`` re-validates ``customFields`` against ``FieldConfig`` ser
- Mongoose schemas set ``strict: true`` so unexpected keys are silently dropped, no
- Sanitize any HTML rendered into ``invoice.template.html`/`purchase.template.html``

Transport & infrastructure

- HTTPS enforced everywhere (Render and Vercel provide this by default – reject a
- CORS locked to the deployed frontend origin only (``CLIENT_URL`` env var), not ``*`
- Helmet middleware for standard HTTP security headers
- Rate limiting (``rateLimiter.middleware.js``) on ``/auth/login`` to slow down brute
- MongoDB Atlas: IP allowlist restricted to Render's outbound IPs where possible
- Uploaded files (signature, logo, scanned purchase bills) validated by MIME type

Financial-data integrity

- Finalized invoices/purchase bills are immutable: the ``/update`` action on a fina
- Invoice/purchase numbering (``Counter`` model) increments inside a MongoDB transa
- Audit fields (``createdBy``, ``createdAt``, ``updatedAt``, and ideally a lightweight

10. Error Handling & Fallback Logging

The goal: no request ever crashes the server unhandled, no error is silently swal

Centralized error flow

- Every controller is wrapped in ``asyncHandler`` (Section 8) so a thrown error – f
- ``error.middleware.js`` is the single place that decides the HTTP response shape: ````js``

```
// error.middleware.js (shape, not final code)
export default (err, req, res, next) => {
  const statusCode = err.statusCode || 500;
  logger.error({
    message: err.message,
```

```

    stack: err.stack,
    path: req.originalUrl,
    method: req.method,
    userId: req.user?.id,
    company: req.user?.company,
  });
  res.status(statusCode).json({
    success: false,
    message: statusCode === 500 ? 'Something went wrong. Please try again.' : e
    errors: err.errors || undefined,
  });
};

```

- A custom `ApiError` class (already in `utils/`) standardizes thrown errors with a `statusCode` and optional field-level `errors` array, so validation errors (400), auth errors (401/403), not-found (404), and server errors (500) are all handled the same way through one middleware — no scattered `res.status(...)` calls in controllers

Fallback logging (server-side)

- Use a structured logger (`winston` or `pino` , already listed in `utils/logger.js`) instead of `console.log` — every log line is JSON with a timestamp, level, and context, so it's searchable later
- Two log destinations at minimum:
 - **Console** (stdout) — Render captures this automatically in its own log viewer, no extra setup needed
 - **Persistent store for anything beyond basic uptime debugging** — once traffic grows, pipe logs to a free/low-cost service (e.g. Better Stack, Logtail, or Sentry for error-specific tracking) since Render's own log retention is short-lived
- Log levels used consistently: `error` (needs attention — failed DB write, PDF generation failure, e-way bill logic exception), `warn` (recovered but noteworthy — retried operation, validation rejection), `info` (business events — invoice finalized, purchase bill created), `debug` (verbose, off in production)
- **Never log**: passwords, JWT tokens, full request bodies containing bank details, or GSTIN/PAN in plaintext logs beyond what's needed to trace the error — mask or omit sensitive fields in the logger's serializer

Fallback behavior for specific failure-prone operations

- **PDF generation failure** (Puppeteer crash, template error): catch specifically in `pdf.service.js` , log the failing invoice/purchase ID, and return a clear "PDF generation

failed, please retry" response rather than a generic 500 — don't leave the client wondering if the invoice itself was corrupted

- **Invoice/purchase numbering conflict** (rare race condition despite the transaction in Section 9): catch the duplicate-key error specifically, retry the Counter increment once automatically before surfacing an error to the user
- **Third-party failures** (Cloudinary upload, future GSP e-way bill API): wrap in try/catch with a timeout, log the failure, and let the core invoice/purchase save succeed independently — a failed attachment upload should never block saving the underlying financial record
- **Database connection loss:** `config/db.js` implements reconnection with backoff and logs each retry attempt; `index.js` doesn't start accepting HTTP traffic until the initial DB connection succeeds

Frontend error handling

- Axios response interceptor (in `axiosClient.js`) catches all API errors in one place: shows a toast/notification with the server's `message`, redirects to login on 401, and never dumps a raw stack trace to the UI
- A top-level React Error Boundary wraps the app so a rendering bug in one page (e.g. a malformed dynamic field) shows a fallback "Something went wrong" screen instead of a blank white page
- Form-level validation errors (from the 400 response's `errors` array) are mapped back to the specific field in the form, not just shown as a generic banner

11. Environment Variables

backend/.env

```
PORT=5000
MONGO_URI=
JWT_SECRET=
JWT_EXPIRES_IN=7d
CLOUDINARY_CLOUD_NAME=
CLOUDINARY_API_KEY=
CLOUDINARY_API_SECRET=
CLIENT_URL=
EWAYBILL_THRESHOLD=50000
LOG_LEVEL=info
```

frontend/.env

```
VITE_API_BASE_URL=
```

12. Deployment Notes

- **Backend → Render:** Web Service, build `npm install`, start `node index.js`. Add all env vars in Render dashboard. Puppeteer on Render needs the `puppeteer` buildpack or switch to `@sparticuz/chromium` + `puppeteer-core` for serverless-safe PDF rendering.
 - **Frontend → Vercel:** standard Vite build, set `VITE_API_BASE_URL` to the Render backend URL.
 - **Database → MongoDB Atlas:** free M0 cluster is sufficient at this scale; whitelist Render's outbound IP or use `0.0.0.0/0` with strong DB user credentials for early-stage, tightened per Section 9 as the client's usage grows.
-

13. Build Order (recommended prompt sequence for Claude Code)

1. Scaffold both repos (folder structure above, package.json, base configs)
2. Backend: Mongoose models (Invoice, PurchaseBill, FieldConfig, Master) → `methods.js` centralized DB layer
3. Backend: Counter/invoiceNumber service (both `INV-` and `PUR-` series) → `gst.service` (shared by both) → `dynamicField.service`
4. Backend: `error.middleware.js` + `logger.js` + `ApiError` / `asyncHandler` wired first, so every subsequent layer has consistent error handling and logging from day one
5. Backend: auth (JWT, bcrypt, rate limiting) → `router.js` wiring all routes to controllers → controllers call `methods.js` + `services`
6. Backend: field-config & masters CRUD (so dropdowns/custom fields exist before invoice/purchase UI needs them)
7. Backend: party/product CRUD → invoice CRUD → purchase CRUD (built right after invoice, same pattern, roles reversed)
8. Backend: PDF generation — sales invoice layout first (matches reference sample)

exactly), then purchase bill layout (same engine, supplier/company roles swapped, plus attachment link) — with the fallback handling from Section 10

9. Backend: build the two remaining templates per document type (`modern` , `detailed`) reusing the same data object, plus `template.controller.js` for selection (Section 16)
 10. Frontend: auth pages → layout shell → Settings screens (Manage Fields, Manage Masters, Template Gallery) first, so later forms can consume them
 11. Frontend: `DynamicFieldRenderer` → party/product CRUD screens using it
 12. Frontend: Invoice creation form with live tax calculation + dynamic fields → invoice list/detail → PDF download button (respecting the company/document template selection)
 13. Frontend: Purchase creation form (reuse invoice form logic, supplier instead of buyer, add attachment upload) → purchase list/detail → PDF download button
 14. E-way bill threshold flag + manual entry screen (shared across invoices and purchase bills)
 15. Reports (sales register, purchase register, combined GST summary)
 16. Security pass: confirm company-scoping on every query, CORS lockdown, Helmet, rate limits, upload validation (Section 9)
 17. Polish: pagination, validation messages, loading/error states, responsive layout, Error Boundary
 18. Deploy backend to Render, frontend to Vercel, connect Atlas, smoke test end-to-end with the reference invoice data
-

14. Out of Scope for v1 (flag to client if raised)

- Direct GSTN e-Way Bill API / e-Invoice IRN generation (needs GSP account)
- Multi-company / multi-branch support (schema is ready for it, UI is not)
- Inventory/stock management beyond product master
- Payment gateway / online collection
- Advanced security hardening beyond Section 9 (e.g. 2FA, IP allowlisting per user, SOC2-style audit logging) — flag if the client's compliance needs require it
- A drag-and-drop custom invoice template builder — v1 ships three fixed, code-defined

15. Frontend Theme & Visual Design System

Explicit goal: this should not read as AI-generated or templated. The palette, type, and layout choices below are deliberately grounded in what this software actually is — a GST ledger/register tool replacing Tally — rather than reaching for the defaults an AI design tool produces by habit. Section notes below call out specifically which common "AI-made" tells are being avoided and why.

The subject matter drives this: the client is replacing **Tally with a GST ledger/register tool**, used daily by an accountant or owner to move fast through repetitive data entry and trust every number on the page. The design should feel like a well-run financial register — precise, quiet, confident — not a generic marketing-style SaaS dashboard with rounded cards and gradient washes. Avoid the common tells of a "templated" look: no warm-cream-plus-serif combo, no near-black-plus-neon-accent, no identical rounded cards with the same soft grey shadow on everything, no ALL-CAPS eyebrow labels or arrow-appended buttons.

Color tokens

Token	Hex	Usage
Ink Navy	#152238	Sidebar, header bar, primary buttons, primary headings
Paper White	#F7F7F5	Main content background — cool, not warm-cream, closer to actual ledger paper
Rule Grey	#D8DAD6	Table hairlines, dividers, input borders — thin, not heavy
Stamp Red	#A8402F	Single accent color — used sparingly for the one thing that needs attention (overdue, e-way bill required, delete action). Referenced from the ink/stamp used on real Indian tax documents, not a decorative brand color
Ledger Green	#2F6F52	Paid/finalized/success states only — never decorative
Slate Text	#3D4451	Body text — not pure black, easier on the eyes for data-dense screens

Use Stamp Red and Ledger Green only for state — never as page decoration or gradient. This is the single boldness point (per the "spend your boldness in one place" principle): everything else stays quiet so the one red or green signal is immediately legible in a table full of numbers.

Typography

- **Headings & nav labels:** a confident grotesk sans with some personality — e.g. *Space Grotesk* or *General Sans* — used at a restrained size, sentence case, never all-caps
- **Body copy & form labels:** a highly legible workhorse sans built for data-dense UI — e.g. *IBM Plex Sans* or *Public Sans*
- **Financial figures (amounts, quantities, invoice numbers, dates in tables):** the **tabular/monospace variant** of the body face (e.g. *IBM Plex Mono* for numerals only, with `font-variant-numeric: tabular-nums` as a fallback) — this is a functional choice, not a decorative "data label" tell: a billing register lives or dies on columns of numbers lining up so a user can eyeball-check totals at a glance
- No serif anywhere — it reads as the generic "AI-editorial" default this brief should avoid, and it fights the tabular precision the content needs

Layout

- Left-aligned throughout — this is a working tool, not a marketing page; center-aligned hero treatments don't belong here
- Fixed dark Ink Navy sidebar (Dashboard, Invoices, Purchases, Parties, Products, Reports, Settings) against the Paper White content area — a real visual anchor, not a rounded floating card
- Tables are true ledgers: hairline row dividers in Rule Grey, no zebra-stripping, no card-wrapping each row, generous but not loose row height so a full day's invoices fit on one screen
- `InvoicePreview / PurchasePreview` intentionally echo the reference sample's own layout logic (boxed header fields, ruled item table, boxed tax summary) rather than reinventing it as a "modern card" — the client and their customers already recognize this layout as a proper tax invoice; changing that visual grammar would undermine trust, not modernize it
- Dashboard uses a small number of real KPIs (outstanding receivables, this month's sales, GST payable) as plain large numbers with a short label underneath — no icon-in-a-circle treatment on every metric, no decorative sparkline unless it's showing genuine trend data

Interaction & motion

- One deliberate motion moment: the invoice/purchase form's live tax calculation updating (CGST/SGST/round-off recalculating as line items change) is the "show what changed" moment worth animating — a brief number roll or highlight flash on the changed total, nothing more
- No fade-slide-up entrance animation on page load, no hover-lift on every table row or card — those are the generic default and read as templated rather than deliberate
- Loading and empty states are written in plain, direct language ("No invoices yet — create your first one" rather than a generic spinner-only state), matching the interface's own voice, not an apologetic tone

Accessibility & quality floor (non-negotiable regardless of theme)

- Visible keyboard focus states on every interactive element (critical here — this is a form-heavy, keyboard-driven data-entry tool)
- Color is never the only signal: a Stamp Red "overdue" state also carries a text label, not just a red dot
- Responsive down to mobile for viewing/reports; full invoice/purchase creation is optimized for desktop/tablet first, since that's how the client actually works, with mobile treated as a read-only/lightweight view
- Respect `prefers-reduced-motion` for the one animated moment described above

16. Invoice & Purchase Bill Templates (multiple layout options)

Note for Claude Code: the exact visual design for each template below will be supplied separately as **reference photos/screenshots** at build time — actual bill formats the client wants replicated (similar to how the Section 2 reference invoice photo defined the Classic Ledger layout and field set). Treat any such image as the source of truth for that template's spacing, rule placement, box structure, and field arrangement — build to match what's in the photo, not just the written description below. If a supplied photo conflicts with the naming/grouping below (e.g. the client provides four distinct designs instead of three, or a design that doesn't cleanly map to "Classic/Modern/Detailed"), follow the photos and adjust the template keys/names to match what was actually provided, keeping the underlying architecture (Section 16's data model, selection logic, dynamic-field rendering) the same regardless of how many or which designs are supplied.

The client shouldn't be locked into one fixed invoice design — like Tally/Zoho Books/Vyapar,

the software should ship with a small set of ready-made templates the client can pick from (per company, with an optional per-document override), and the dynamic fields from Section 6a should render correctly inside whichever template is active.

Template set for v1 (three, not a large gallery — keep scope tight; names below are placeholders until reference photos are supplied):

1. **Classic Ledger** (default) — matches the reference sample's own layout exactly: boxed header grid, ruled item table, boxed tax summary block, bank details footer. This is what the client's customers already recognize as "a proper tax invoice," so it stays the default and the most complete implementation.
2. **Modern Compact** — same field set and same GST-compliance structure, but tightened spacing, a cleaner single-rule table (no boxed grid around every header field), and the company letterhead treated as a simple top band — for clients who print less and email PDFs more.
3. **Detailed Register** — for high-line-item invoices/bills (10+ items): denser row height, item table repeats its header on page breaks, tax summary stays pinned to the last page — built for volume, not aesthetics.

All three carry identical statutory content (Section 2 fields, GST logic, HSN breakup) — they differ only in spacing, rule weight, and information density, never in what's legally required to be present.

Architecture

- `Company.invoiceTemplate` and `Company.purchaseTemplate` — the company's default choice (`classic` | `modern` | `detailed`)
- `Invoice.templateOverride` / `PurchaseBill.templateOverride` (optional) — lets a specific document use a different template than the company default, without changing the company-wide setting
- Backend `templates/` folder holds one HTML file per template per document type:

```
templates/
├── invoice/
│   ├── classic.template.html
│   ├── modern.template.html
│   └── detailed.template.html
└── purchase/
    ├── classic.template.html
    ├── modern.template.html
    └── detailed.template.html
```

- `pdf.service.js` **resolves** `(documentType, templateKey)` → template file, then injects data + any active `FieldConfig` custom fields in their configured `displayOrder` — the same data object feeds every template, so adding a custom field once makes it appear in all three layouts automatically
- Templates are cached in memory per Section 8's optimization note — read once at boot, not per request

Frontend

- `Settings → Invoice Template` and `Settings → Purchase Template` pages show a small gallery: a thumbnail/preview image of each of the three templates with a "Select" action, plus a live preview pane rendering the client's own real company details and the reference sample's sample data inside the chosen template, so they see exactly what will print before committing
- `InvoicePreview.jsx` / `PurchasePreview.jsx` accept a `templateKey` prop and switch their on-screen rendering to match — the on-screen preview and the server-generated PDF must stay pixel-consistent, so both read from the same field-order/layout rules rather than the frontend guessing independently
- New components: `components/settings/TemplateGallery.jsx`,
`components/settings/TemplatePreviewCard.jsx`

Endpoints

```
POST    /api/v1/templates/list           # body: { documentType } — invoice
POST    /api/v1/companies/set-template # body: { companyId, invoiceTempla
```

(No separate CRUD needed for templates themselves in v1 — the three are fixed, code-defined layouts; only the *selection* is dynamic. Building a true drag-and-drop template editor is a larger v2 feature — flag to the client if they want that level of control, and add it to Section 14's out-of-scope list.)